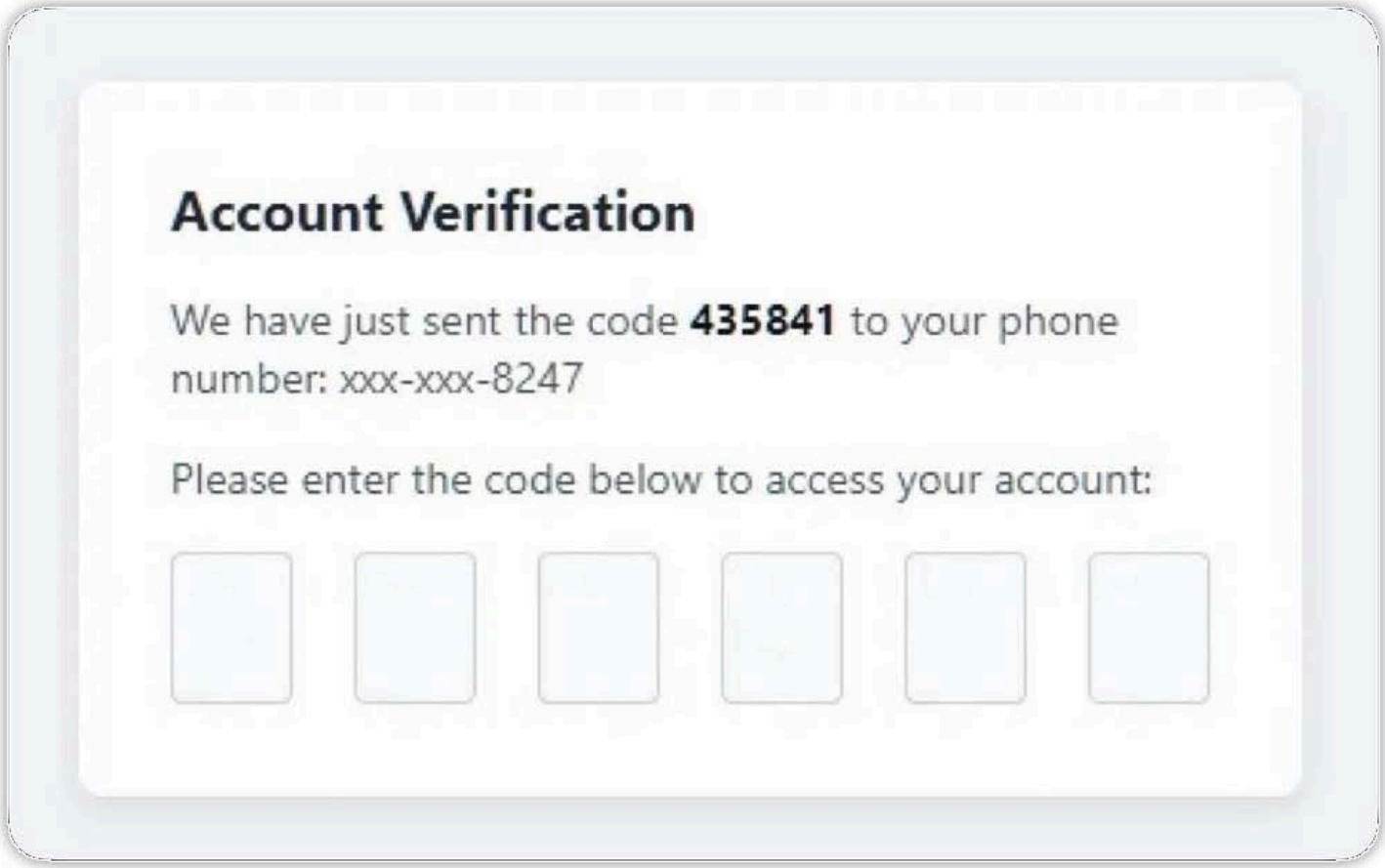
Security Now! #1059 - 01-06-26

MongoBleed

This week on Security Now!

• Code-signing certificate lifetimes shortened by two years. • Sadly, ChatGPT is heading toward an advertising profit model. • The Python Package Index is strengthening its security. • BitLocker gets hardware acceleration, but not today. • New York City's mayoral inauguration banned Raspberry Pi's. • An astonishingly good British time travel series. • A critical link between Vitamin D and Magnesium. • A look inside the very bad MongoBleed vulnerability.

"Why reinvent the wheel? Allow Agentic AI to take all the drudgery out of your repetitive coding tasks!!"



Security News

Code-signing certificate lifetime dropping from 39 months to 15.
I want to begin this first podcast of 2026 by exploring around the edges of a recently decided, announced and somewhat discouraging update that follows a disturbing trend which will have a significant impact upon our industry. I understand that those behind it are claiming it will have a net positive impact upon security, but I question whether that's true and I suspect that the positive impact it will most have is upon certificate authorities revenue and profits.

Today's level of persistent cybercrime is pushing the world to a place where only software that is validly signed will even be considered for execution. Signatures are now required for iOS apps, Linux distros, Secure booting, Android APKs, Browser extensions, all of the various gaming consoles, SmartTVs, and firmware for home routers, NASes and even automobiles. All of these need to be signed. Linux, being inherently more open, is the only remaining OS where signing is either unnecessary or not strongly needed. Windows apps can theoretically run without signing, but only if they are very well known. The only hope any newly minted Windows app has of running today is if it's carrying a signature and even then, only if that signature itself has previously established a strong reputation by virtue of the applications it has signed. GRC is now able to produce and sign Windows apps on the fly with very little trouble. But we've seen that other apps, such as Notepad++, which have a sterling reputation will have serious trouble if they are unsigned or, as its author briefly attempted, are self-signed, which means nothing in practical terms since anything can be signed with a self-created certificate. If Linux is lax and Windows is quite guarded, then macOS sets the bar about as high as it can go. Any macOS application that's not signed is assumed to be malicious. You really need to be a registered developer in good standing to have any chance of a macOS running your software.

That's pretty much where we are today. Essentially, anywhere it's practical to require a signature on software, a signature will be required. The problem is, this is still an imperfect system. Bugs in signed software are no less prevalent than in unsigned software. So signing offers no guarantee about software quality, and bad guys are just as able to exploit bugs in signed, as in unsigned, software. But it is certainly worthwhile to require a signature rather than not. If nothing else, something somewhere is known about the signer of the software. There is at least some modicum of accountability and traceability. And if a piece of signed software is discovered to be malicious, its signing certificate can be black-listed so that nothing else signed by that presumably malicious certificate will be trusted.

It's not unreasonable to expect a Linux user to be cautious about what and where they obtain the software for their machines. But that's not the case for the casual Windows user who browses around Microsoft's Windows store looking for stuff to download and run. So everything and anything that comes from the Windows app store is signed by a known developer.

We're talking about what has become the crucial security topic of code signing today because, in another move that makes very little sense to me, late last year the CA/Browser forum voted to reduce the maximum lifetime of code signing certificates, for any certificates issued from March 1st of this year, less than two months from now, from 39 months, which is a comfortable 3 years plus 3 months, to a far less convenient 1 year and 3 months. And this is occurring for no apparent reason that addresses no apparent problem.

Back in 2022 the policy was finalized that no code signing private keys could exist outside some form of hardware token or HSM which would prevent their theft. That policy took effect on June 1st, 2023, two and a half years ago. From that date on, certificate authorities would only issue code signing certificates in hardware. And,. critically, this applied not only to extended validation code signing certs which had always been required to reside in hardware isolation, but to all

code other lesser-verified code signing certificates. This move, made two and a half years ago, ended the opportunity for code signing certificates to be remotely stolen. This means that the only possible attack on them would be physical and local. For the past two and a half years, no code signing certificate could be exfiltrated by any remote attacker. Nevertheless, the certificate authorities have voted and decided that even safely stored code signing certificates must be renewed much more frequently.

The shorter lifetimes introduce another wrinkle as well, which is that unlike with the ACME protocol, that's able to completely automate the validation, issuance and installation of TLS certificates into their hosting servers, there's no automation available to install new code signing certificates into hardware security modules. It doesn't exist.

This raises the question: Why would the CA/Browser forum feel the need to reduce the life of absolutely theft-proof code signing certificates? What benefit could there be to them? And does this have any impact upon the "browser" side of the CA/Browser forum? Looking over the results of the ballot measure CSC-31 titled "Maximum Validity Reduction" I was struck by the mix of voters – and using the term "mix" would be inaccurate, since all ten of the "Yes" votes came from certificate issuers.

While updating myself about what's been going on and poking around the industry I stumbled upon an interesting tidbit that pretty much explained what's happening. The lightbulb lit. There's been a recent significant increase in cloud-based code signing. In other words, the push for shorter and less convenient use of super-secure hardware security modules, by shortening the maximum life of the certificates they can obtain and store, while providing no automation for their management, has the indirect effect of actively discouraging code signers from obtaining and managing their own code signing certificates. It appears to be that the future of code signing will be the establishment of a subscription relationship with a major provider such as GlobalSign or DigiCert.

Remember that what code signing actually signs is a cryptographically secure hash of some code. This makes it entirely feasible for that process to be "remoted" with a cloud-based service. A cloud-based code signing utility takes a cryptographic hash of the code to be signed and forwards it to the signing provider's cloud service. After verification and validation of the identity of the signing party – and, of course, after first verifying that their subscription is in good standing and they're all paid up – the cloud signing provider uses the customer's own private key – which the provider maintains for them and their customer never receives or sees – to sign the hash of their code. The signed hash is then returned to the customer whereupon the cloud-signing utility affixes it to the end of their code to complete the signing process.

So taken in aggregate, what has happened, and believe me this is deeply disturbing, is that to an ever increasing degree, all code from anyone and anywhere is inherently mistrusted by default – it will probably only run on Linux – unless it has been signed by one of a diminishing number of increasingly large select few signers who are pretty much then free to charge whatever they wish for the privilege. That's right. What has slowly been growing and evolving is a cabal. We've been witnessing a consolidation of certificate authorities over the past decade as the bigger fish swallow up the littler fish while also, not surprisingly, raising their prices.

Today, the least expensive code signing certificate I could locate was IdenTrust's at $270/year, but purchasing a 3-year certificate offers a 20% discount, so that's $647 for 3 years. GlobalSign is just over twice as expensive per year at $550/year with no multi-year discount, and DigiCert leads the pack at $840/year. Think about that for a second. $840/year, for no reason other than because they can.

All the commercial platforms now require code to be signed and a very small and shrinking group of increasingly powerful commercial authorities have decided to follow the TLS model of continually shortening the lifetime of those code-signing certificates, which they alone are empowered to issue. Today's code must be signed, so code authors have no recourse other than to pay an annual tribute to the certificate gods in order to qualify for the privilege.

It's against this backdrop that the certificate authorities all voted to take two full years off of the maximum code signing certificate lifetime, reducing it by 24 months from 39 months to just 15 months. Why? Because they can. Because there's no one to stop them. Certificates that have been locked up in hardware are not subject to remote attacker theft. And we know where this is headed, right? We've seen this play out already once with web server TLS certificates. We've watched as TLS certificate lifetimes gradually dropped from their original lifetime of 10 years and are now headed down to 47 days a few years from now. With certificates expiring more often than every 7 weeks, automation is the only practical solution – despite all of the many inconveniences it incurs in situations where use of the ACME protocol is not practical.

And so the same thing is clearly happening with code signing. Once the various certificate authorities get the infrastructure in place to support cloud-based code signing, that'll be the only practical way code can be signed. Maximum code-signing certificate life was just reduced for no reason, effective this coming March 1st. Does anyone imagine that will be the end of it? In the future, it will be necessary for anyone who wishes to produce software for general use that any platform will accept, to obtain and maintain an account with a cloud-based code signer. I don't know when it will happen. It'll likely be gradual over time. Just like it was with TLS certs. But no one needs to wait for that day. DigiCert is ready today. For only $1104/year they'll be glad to sign your code in their cloud. But there is one glitch, they limit it to 1,000 signatures per year. I'm not kidding. Unlike the past world where, after obtaining a code signing certificate, we were free to exercise our right to sign as much code as we like, once code signing has evolved into a dis-service, the provider will hold not only our private key, but all of the cards. And with ever-fewer certificate authorities we can expect this to continue increasing in cost.

So, for what it's worth, anyone who has been signing their own code, or who may be getting ready to make that jump, might wish to grab a 39-month code signing certificate while you still can, prior to March 1st. You'll be able to obtain an additional two years of hassle-free, cloud-free and also unlimited code signing. Thanks to our listeners' generous purchases of SpinRite and our new DNS Benchmark – all of which is signed – I can afford to take my own advice and I plan to do so. Needing to continue updating a hardware security module, while not a big problem, should not be necessary. There's no big problem that's being solved by shortening the lifetime of any certificate that's stored in hardware. So forcing this upon the world appears to be about nothing more than profit and control. Because they can.

I'm sure our listeners are also aware that none of our real world experience suggests that the use of a remote 3rd-party cloud-based signing system would be more secure than simply signing with the use of a local physical dongle that can remain offline until needed. Earlier, I glossed over the fact that the remote cloud-signing service will need to be certain that the certificate's owner is the one requesting that some code be given their signature. I don't know that I want my private code-signing key to be held in the cloud by any 3rd-party. How is that more safe?

And before we leave the topic of certificate lifetimes, I'll remind everyone of the upcoming March 15th which is also approaching. That's when maximum TLS certificate lifetimes will be cut in half, from around 398 days to just 200 days. So anyone who may need to be managing TLS certificates manually, and there are many such use-cases, updating close to, but before, this coming March 15th deadline will allow you to defer the need to find some better solution for another 13 months.

ChatGPT heading toward advertising support

We were talking not long ago about the sad idea that ChatGPT's clean "answers only" dialog might become laden with advertising. Anyone who was around during the birth of "The Google" will fondly recall the original super-clean no-nonsense Google search results. Those days died once Google realized how much money could be made through advertising. One of the observations I can't help making is that AI is currently a money-losing enterprise with high hopes for the future. That's worrisome because I, like many others I'm sure, have now figured out what our current AI is, and how to leverage its benefits. I don't ever want to lose access to it.

A couple of days after Christmas, Tom's Hardware posted reporting along these lines which I wanted to share because it contains a bunch of additional interesting detail. Their headline was: *"ChatGPT could prioritize sponsored content as part of ad strategy"* They open by posing the rhetorical question "*Are we going to see ads in ChatGPT's answers soon?"* and they explained:

> *OpenAI is allegedly still working on adding ads to ChatGPT, with sources saying staff are discussing ways to bake them into the chatbot's responses. According to The Information, the AI company is looking to create a new type of digital ad rather than simply copying what existing search and social media companies are running. This is possible because OpenAI can use historical chat data to serve ads that are highly relevant to users' interests.*

I'll interrupt to note that it's difficult to argue with that. I mentioned that many of us have come to understand what's going on with LLMs and we understand the "context window" that an account holder builds over time. I was a bit taken aback the first time ChatGPT offered some example code to me in Intel assembly language. I was quite certain that wasn't what it would have offered most others. But I've come to appreciate the degree to which it is able to tune its replies to our dialog history.

Now, this is not to say that it's going to have any advertisers in its bag that will match up with my interests, but I certainly understand the notion of an AI that has been working with someone for a while being unusually well suited to matching them with relevant advertisers. That idea has clear merit. And we know from our previous study of advertising, tracking and profiling that much more accurate matching means much more revenue for every highly targeted ad. Tom's continues, writing:

> *OpenAI told The Information: "As ChatGPT becomes more capable and widely used, we're looking at ways to continue offering more intelligence to everyone. As part of this, we're exploring what ads in our product could look like. People have a trusted relationship with ChatGPT, and any approach would be designed to respect that trust."*
>
> *Staff discussions on ad implementation have ranged from prioritizing sponsored content in the chatbot's answers to adding a sidebar that shows ads related to the user's query. They've also considered showing them only when the conversation moves toward shopping or similar activities, or as a secondary step where ads are displayed only when someone clicks a link in ChatGPT's results.*
>
> *It's been reported that OpenAI is shifting its focus away from ads, especially after CEO Sam Altman declared a 'Code Red' for the company following the latest version of Google's Gemini, which outpaced ChatGPT in several benchmarks. Altman said that OpenAI needed to improve the AI chatbot's personalization, speed, and reliability, and cover a broader range of topics, so the company is pausing work on all other projects to focus on these capabilities. However, it*

seems to be continuing progress on ChatGPT ads, despite the recent change in focus.

ChatGPT has three main revenue streams at the moment — subscriptions to ChatGPT Plus, Pro, and Business, API access for developers, and enterprise solutions. Aside from that, OpenAI said it will start earning revenue from non-paying users by 2026, projecting $2 per user annually, which will grow to $15 per year by 2030. Despite that, OpenAI has yet to turn a profit since its founding in 2015. Even though its annualized revenue hit $10 billion earlier this year, it's still expected that the company's operating losses will hit $74 billion annually by 2028. Nevertheless, investors continue to pour money into the company, even as some are starting to ask how its long-term profitability will look.

For comparison, Google's ad business accounted for $237.8 billion in revenue in 2023, representing 77% of the company's total revenue. This amount is more than enough to cover OpenAI's estimated losses, and it seems it wants to follow the search giant's playbook by baking ads into its results as well. However, this also raises privacy concerns, especially since ChatGPT likely has much more information about its users than Google does. Furthermore, there's the question of how OpenAI will ensure its LLM gives the best answer to the user, especially if it stands to make money by showing ads instead of organic results.

Yes. No one wants a skewed reply from an AI that's trying to lead its user down one commercial path because of a hidden kickback that the AI receives.

PyPi increases security

There's some good news, finally, on the Python Package Index (PyPI) repository front. PyPI posted in their "PyPI in 2025: A Year in Review" the following, which which I have excerpted:

As 2025 comes to a close, it's time to look back at another busy year for the Python Package Index. This year, we've focused on delivering critical security enhancements, rolling out powerful new features for organizations, improving the overall user experience for the millions of developers who rely on PyPI every day, and responding to a number of security incidents with transparency. Let's look at some numbers that illustrate the sheer scale of PyPI in 2025:

- *More than 3.9 million new files published*
- *More than 130,000 new projects created*
- *1.92 exabytes of total data transferred*
- *2.56 trillion total requests served*
- *81,000 requests per second on average*

Yikes! An average of 81,000 requests per second. That really does give some sense for the scope and scale of today's repositories, and it becomes clear just how rapidly a popular package, if its developer's account can be compromised, has the ability to spread. I recall when the notion of a *"supply-chain attack"* was a new term and concept for us on this podcast. Now, sadly, it has become one of the most prevalent and worrisome. Their posting noted:

These numbers are a testament to the continued growth and vibrancy of the Python community. Let's dive into some of the key improvements we've made to PyPI this year.

Security First, Security Always: Security is our top priority, and in 2025 we've shipped a number of features to make PyPI more secure than ever: Enhanced Two-Factor Authentication (2FA) for Phishing Resistance: We've made significant improvements to our 2FA implementation, starting with email verification for TOTP-based logins. This adds an extra layer of security to your account by requiring you to confirm your login from a trusted device, when using a phishable 2FA method like TOTP. Since rolling out these changes, we've seen more than 52% of active users with non-phishable 2FA enabled.

And what we see on this podcast is evolution. Recall when the concept of a continually-changing 6-digit code was going to be the *"end all be all"* of security? It was exciting because even if some site were to lose control of its static passwords, no bad guy would be able to produce the "one in a million" 6-digit code that was correct and changing every 30 seconds.

It was a nice theory while it lasted. But then reality struck. We learned that practical applications of time-based one time passwords actually needed to open a surprisingly large acceptance window for codes. They needed to accept many minutes worth of code on either side of the optimal code in order to minimize false negative failures caused by desynchronized clocks, or communications delays, and maybe even operators cutting and pasting or emailing themselves the codes, or lord only know why.

But the real death knell sounded when the bad guys realized that those larger acceptance windows meant that users could be readily phished by having them attempt to log into a fraudulent website which they might get to by clicking a link in an email. They would provide their username and password and then be prompted for their one-time password. The bad guys would collect all of that and log into their actual account on their behalf. And when that might be a corporate VPN or remote access portal, much damage could result.

So what the PyPI folks are saying is that *"Sure, by all means use two factor authentication. But so many of our past PyPI package submitter's accounts have been hacked, even when they were "protected" by TOTP one-time passcodes, that we are strongly urger all developers to allow us to require that they also, on top of all that, respond to a link emailed to their account's registered email address."* The requirement of an email loop authentication slows down the whole login process, but the demonstration of control over an email account remains a strong, useful, and intuitive authentication factor which displays every sign of being with us for many years to come. It's great news that PyPI is actively working to strengthen their authentication.

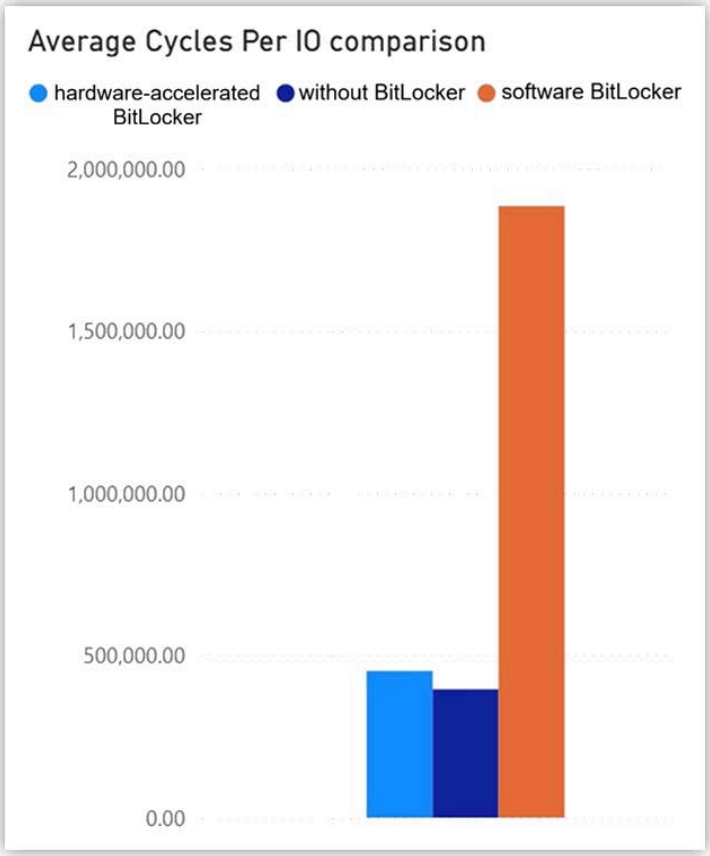
BitLocker to get hardware acceleration support under Windows 11.

Shortly before Christmas, Microsoft's Windows IT Pro Blog posted the news that Windows 11 would be adding support for hardware encryption and decryption to their BitLocker whole-drive encryption system. The chart of the relative performance of no BitLocker compared to software crypto vs hardware crypto is quite bracing, but first let's see what Microsoft explained. They wrote:

We know that users desire both security and great performance. Historically, we have strived to keep BitLocker performance overhead within single digit percentage points. However, with the rapid rise in popularity and advancement of Non-Volatile Memory Express (NVMe) drive

technology, these drives now achieve much higher Input/Output (I/O) operation speeds. As a result, corresponding BitLocker cryptographic operations can require a higher proportion of CPU (Central Processing Unit) cycles. This makes the performance impact of BitLocker more pronounced, especially on high-throughput and I/O intensive workloads like gaming or video editing.

Right. In other words, there's a fixed absolute overhead cost that's required to encrypt and then later decrypt all of the blocks of data being written to and read back from non-volatile mass storage. The cost is a fixed function of processor speed and the amount of data being read and written. Significantly, it is entirely independent of the storage medium being written to or read from. Microsoft talks about the overhead as a percentage that's added to the time that would be required without BitLocker. That's certainly a reasonable way to view the encryption overhead.



But then along comes these pesky super-fast NVMe drives which are essentially PCIe devices themselves. SATA drives used a SATA interface to a SATA controller which was then attached to the processor's PCIe bus. But NVMe drives need no controller. They are themselves first-party PCIe devices. So they're able to stream their data at the highest speed possible directly to and from the rest of the system.

What this means in practice is that someone inside Microsoft realized that the actual delivered performance of NVMe drives was now being dramatically limited by the fixed-speed overhead introduced by BitLocker. The chart above-right, kindly provided by Microsoft, demonstrates the significance of the encryption overhead. The shortest center bar shows the average CPU cycles per IO operation. The hugely tall orange bar is the average number of CPU cycles incurred by software BitLocker encryption. It's quite sobering, standing about four times the height of the no encryption bar. And finally, by comparison, the hardware-accelerated BitLocker only adds a modicum of additional overhead to the transfer.

The very clear takeaway from this is that anyone who is currently using BitLocker on an NVMe drive **without** the benefit of Windows 11's forthcoming BitLocker hardware encryption is seeing only a true fraction of the performance they could be obtaining from that drive without the comparatively massive overhead that's being introduced by BitLocker.

Someone's probably wondering what about SpinRite? So I'll just mention that you get 100% full

performance with SpinRite, regardless of whether BitLocker is present or not, because SpinRite 6.1 doesn't bother with BitLocker encryption and decryption. It just works on the raw encrypted data. But when you actually need to read, write, understand and use the drive's data, as Windows does, then there's no choice other than to run through BitLocker's crypto pipeline.

Since the performance with and without BitLocker, and with and without hardware acceleration is pretty astonishing, let's see what more Microsoft has to tell us about this. They continue:

As NVMe drives continue to evolve, their ability to deliver extremely fast data transfer rates has set new expectations for system responsiveness and application performance. While this is a major benefit for users, it also means that any additional processing — such as real-time encryption and decryption by BitLocker — can become a bottleneck if not properly optimized. For example, professionals working with large video files, developers compiling massive codebases, or gamers demanding the lowest possible latency may notice delays or increased CPU usage when BitLocker is enabled on these high-speed drives.

Balancing robust security with minimal performance impact is more challenging than ever. The need to protect sensitive data remains critical, but users also expect their devices to operate at peak efficiency. As a result, the industry has needed to innovate new solutions that ensure both security and speed are maintained even as hardware capabilities advance.

To achieve this, we announced hardware-accelerated BitLocker at Microsoft Ignite last month. Hardware-accelerated BitLocker is designed to provide the best combination of performance and security.

Starting with the September 2025 Windows update for Windows 11 24H2 and the release of Windows 11 25H2, in addition to existing support for UFS (Universal Flash Storage) Inline Crypto Engine technology, BitLocker will take advantage of upcoming system on chip (SoC) and central processing unit (CPU) capabilities to achieve better performance and security for current and future NVMe drives.

These capabilities are:

- *Crypto offloading – BitLocker shifts bulk cryptographic operations from the main CPU to a dedicated crypto engine. This capability frees up CPU resources for other tasks and helps improve both performance and battery life.*

- *Hardware protected keys – BitLocker bulk encryption keys, when necessary SoC support is present, are hardware wrapped, which helps increase security by reducing their exposure to CPU and memory vulnerabilities. This is an addition to the already supported Trusted Platform Module (TPM), which protects intermediate BitLocker keys, putting us on a path to completely eliminate BitLocker keys from the CPU and memory.*

When enabling BitLocker, supported devices with NVMe drives along with one of the new crypto offload capable SoCs will use hardware-accelerated BitLocker with the XTS-AES-256 algorithm by default. This includes automatic device encryption, manual BitLocker enablement, policy driven enablement, or script-based enablement with some exceptions. (See the Enablement and management experiences section for more details.)

We have enhanced the architecture and implementation of the Windows storage and security stacks to support these new capabilities as an operating system enhancement that will bring

value to all capable PCs over time. Upcoming Intel vPro® devices featuring Intel® Core™ Ultra Series 3 (formally codenamed Panther Lake) processors will provide initial support for these capabilities with support for other vendors and platforms planned. Coordinate with your suppliers and keep an eye on listings from us and other vendors as PCs become available on the market.

Huh. So... all of this fancy new BitLocker crypto-engine pipeline support will only be available when using these next generation Intel processors which were just unveiled by Intel yesterday during CES – the annual Consumer Electronics Show. This means that regardless of the version of Windows being used, 7, 8, 10 or even the latest 11, on our current hardware, the use of BitLocker is exacting a tremendous performance penalty that Microsoft is only now disclosing now that they have a solution. But that solution is tomorrow and not today.

I'd say that if anyone has a BitLocker-encrypted NVMe drive which they encrypted out-of-the-box just because "why not?" where their operating environment doesn't really require encryption, and where they'd rather receive a huge boost in performance, it might be worth considering de-BitLockering any high speed NVMe drives, reducing the load on your processor, improving the real-time performance of everything else, and finally obtaining the full performance that's available from a state-of-the-art NVMe drive.

Everything that Microsoft wrote about the increased overhead of fixed-speed encryption and decryption in light of the new faster performance of NVMe drives makes absolute sense. What might also make absolute sense is waiting until your machine's hardware is able to support ultra-low overhead BitLocker encryption unless having it now is really necessary.

Microsoft ended their post by showing how anyone could check to see whether a BitLockered drive's encryption was hardware accelerated. They wrote:

To check if your device is using hardware-accelerated BitLocker, open a command prompt as an administrator and run "manage-bde -status". Look at the Encryption Method section — if "Hardware accelerated" is shown, it indicates that BitLocker is utilizing the SoC's crypto acceleration capabilities.

```
PS C:\Users\AdminUser> manage-bde -status
BitLocker Drive Encryption: Configuration Tool version 10.0.10011
Copyright (C) 2013 Microsoft Corporation. All rights reserved.

Disk volumes that can be protected with
BitLocker Drive Encryption:
Volume C: []
[OS Volume]

    Size:                 127.68 GB
    BitLocker Version:    2.0
    Conversion Status:    Used Space Only Encrypted
    Percentage Encrypted: 100.0%
    Encryption Method:    XTS-AES 256 (Hardware accelerated)
    Protection Status:    Protection On
    Lock Status:          Unlocked
    Identification Field: Unknown
    Key Protectors:
        TPM
```

No one is going to see that today but this is a useful tip for the future when you're running Windows 11 on newer hardware that may be able to offer this support.

Many years ago, back when we were talking about and exploring whole-drive encryption with TrueCrypt, I clearly recall wondering about the performance overhead of using it. So I did some benchmarking of a system's read and write performance with and without TrueCrypt. I recall being surprised that I was unable to detect any performance overhead being introduced by its on-the-fly encryption and decryption. And while I no longer recall the specifics, it's likely that the system I was using back then had a fast processor and a comparatively slow spinning hard drive. So the overhead that was being introduced by the encryption and decryption would have been completely masked by the drive's physical read and write performance that would have been slower than that system's encryption rate.

What's changed is that we now not only have solid state mass storage as the new default, but that storage is being attached directly to the system's I/O busses to deliver unprecedented performance. Software-based encryption and decryption cannot keep pace.

The New York City mayoral inauguration's banned items list

Last week, the newly elected and controversial mayor of New York City was inaugurated. That's not an event that would normally be mentioned here. But this inauguration was a bit special. I'm going to deliberately keep those who haven't already heard about this in suspense for just a minute because the reveal of this is just too much fun. The reporting that I want to share over this is from a perfect perspective and by someone who writes quite well. They wrote:

> *Public safety rules should be dull in the best possible way. Clear. Predictable. Written by people who understand what actually causes harm in a crowd of thousands. New York City usually gets this right. It has decades of muscle memory for doing hard things in public, under pressure, without panicking. Which is why the prohibited items list for the January 1, 2026 NYC mayoral inauguration block party seemed… off.*

At this point the blog post provided a link to the list of prohibited items, which I'm going to share with our listeners. The notice read:

> ***Prohibited Items*** *All spectators will be screened as they enter the viewing area. The following items are prohibited: Large bags, Weapons, fireworks or explosives, large backpacks or duffle bags, drones or remote-controlled aerial devices, strollers, coolers, chairs, blankets, umbrellas, beach balls, bicycles or scooters, alcoholic beverages, illegal substances, pets (other than service animals), large items that could obstruct views of spectators around you, laser pens, bats and batons...*

And finally, tacked onto the bottom of the list as the final two items... what do we find? Flipper Zero and Raspberry Pi. Yep. We wouldn't want any of those crowd-disturbing technologies or capabilities being bandied about carelessly. The posting to the blog of the well-known and very popular AdaFruit website continues:

Explicitly banned: Raspberry Pi and Flipper Zero. WHY? Not categories. Not capabilities. Two named devices. Brand (TM) names. Parked right next to weapons, explosives, and drones, as if the list itself is supposed to do the thinking for us. Raspberry Pi is a general-purpose single-board computer. It shows up in classrooms, newsrooms, accessibility rigs, art installations, and civic tech demos.

Flipper Zero is a consumer electronics testing tool, but its functional territory overlaps heavily with laptops, smartphones, radios, and microcontrollers that remain perfectly legal to carry. If the concern is electronic interference, signal disruption, or hacking, the policy does not say that. It gestures vaguely by naming a couple of gadgets and hoping the implication sticks. Curiosity is now contraband.

There already is a list of prohibited items that works great. At Times Square on New Year's Eve, one of the most tightly secured public events on the planet, the prohibited list is blunt and practical. Backpacks. Drones. Weapons. Alcohol. Large objects that block movement or sightlines. The rules focus on crowd dynamics and physical risk. They do not play whack-a-mole at the end with brand-name electronics.

When a policy bans specific devices rather than behaviors or capabilities, it creates ambiguity for people on the ground. Once a Raspberry Pi is banned, a smartphone sails through security despite being way more powerful, more connected, and more capable of surveillance, disruption or both! That is not a security framework. That is a "vibe" based list, maybe it was AI-generated? That would be interesting if that is what happened.

If the goal is to restrict electronic interference, the language should say so plainly. Unauthorized transmitters. Signal interception tools. Electronic hacking devices. Those are enforceable things already. Naming a short list of familiar gadgets reads less like safety planning and more like anxiety fossilized into policy.

There is a cultural cost to banning brand names like Raspberry Pi. New York is full of educators, artists, technologists, and journalists who use small embedded computers as tools of expression and access. A device-specific ban turns curiosity itself into something suspicious, while ignoring the far more capable computers already in everyone's pockets. The future ban list will have everything!

Today it's Raspberry Pi and Flipper Zero. Tomorrow it's BeagleBone Blacks, Arduino Qs, ESP32 dev boards, Teensy boards, Pine64s, Orange Pis, Jetson Nanos, USB logic analyzers, SDR dongles, Bus Pirates, Defcon badges, hotel keycards, garage door openers, Tamagotchis, graphing calculators, old Nokias, Game Boys with link cables, a TI-83 held sideways, a Pocket Operator making beeps too abrasively, a Furby with unresolved father issues, and some guy's wristwatch that definitely has a microcontroller in it. Meanwhile, everyone walks through holding a smartphone that can film, scan, transmit, triangulate, and livestream the entire event in 4K.

I tried to find emails to someone on the Mayor's team, DMed their socialz, etc. So far here's what I received from the Mayor's team (and some auto-replies/bounces): "Looping in Audra Heinrichs to help answer your questions! All the best, Penelope Birnbaum" Penelope was the press assistant on Kamala Harris' presidential campaign, and now Press and Digital Associate, Zohran for NYC. Audra Heinrichs "directed all press logistics on the Mamdani campaign's final events.

Public safety is a beacon, a flashlight, not a fog machine. They have heavy hitters, they can fix this. The list feels symbolic rather than functional. New York has done better before, and it can do better again. There is enough time for the new mayor's team to check this out, and if they do, I'll get the word out and say there will be no tickets to a security theater.

I can kinda see the Flipper Zero being on the list, and I can see why it might have needed to be directly named. It's a now-famous and mischievous hacking tool that has no real place or purpose at such a public event. If someone were to attempt to bring one in, it would not be unreasonable to ask them why, and then probably to hold it for them until they were leaving afterward. And it really would need to be called-out by name since using the generic "no transmitters allowed" would rule out anyone bringing their phone.

That said, I totally agree that the addition of the Raspberry Pi to the list is nothing short of nuts. And I can see why this author wondered whether AI may have had a hand there. Although it's all academic now, it would be interesting to know exactly where those last two items came from.

Sci-Fi

Forbes headline was: *"Netflix's Best New Show Has A 100% Rotten Tomatoes Score, But There's A Catch"*. They are describing a two-season, 16 episodes in total, astonishingly well conceived science fiction time travel series that can currently be found on Netflix and AppleTV. Amazon Prime Video now only has season 2. It's called *"The Lazarus Project"*. There's a movie by the same name, and "Lazarus" has been used several times before. So beware of name collisions when you're searching. The one I'm talking about is a two season British television series. I was made aware of it when it popped up on Netflix with the news that it would be leaving a few weeks from now, on January 28th. I don't know whether or when AppleTV and Amazon may be losing it. But since I never want to be without it, after getting a couple of episodes into the second season I immediately purchased both seasons on AppleTV, which I assume means I'll always have access to it.

Following their headline: *"Netflix's Best New Show Has A 100% Rotten Tomatoes Score, But There's A Catch"*, they wrote: *"That show is The Lazarus Project, a sci-fi series that originally aired in 2023 on Sky, but has now ported over its two seasons to Netflix. The series has a perfect 100% score on Rotten Tomatoes from critics, an infrequent feat."* I checked over on Amazon Prime where it has a 4.8 out of 5 with most giving it 5 stars and few 4 stars. No one rating it 1, 2 or 3 stars. So I'm mystified by the show's comparatively low 7.3 rating on IMDB because I have never, and I really mean never, seen a more compelling, astonishingly clever and gripping time travel concept and plot. The Lazarus Project is truly remarkable science fiction. This show is so good that I felt duty bound to tell everyone here. And I also posted about it in GRC's Sci-Fi newsgroup. One of the denizens who hangs out over there replied: *"I watched it somewhere besides Netflix and I have to admit I was amazed as well. But I strongly recommend that people binge-watch it, because the plot is highly complex and some critical plot points happen almost in passing. This is not a series to watch in the background while you're doing something else. The logic of the time resets will have you twisted in knots at times. But it's a completely new take on time travel."*

I replied to Milton's posting, writing: *"I agree 100%. It takes extreme attention and focus, which is part of what makes it so good. It's the absolute reverse of "nothing much happened during that episode." The sense is that they're working to cram as much content into each episode as possible. And they succeed."*

The series has been nominated for a BAFTA award. BAFTA is the British Academy of Film Awards, which is Britain's highest honor for British cinema. There is one down side, which is the catch that Forbes referred to in their headline: It's that the series apparently did not plan to end after just two seasons. It proved to be a bit too much for Sky TV's British viewer demographic, which I can understand, since it really is quite a lot. So Sky chose not to commission a third season and we're left hanging. But PLEASE (please please please!) don't let the lack of a nicely wrapped-up with a bow on top conclusion, put you off. What's there is truly wonderful. If you already have a Netflix subscription it's really a no-brainer. It's sitting right there just a few clicks away, and it's astonishingly mind blowing in every right way. If you don't have Netflix but you do have AppleTV, you could purchase just the first episode of the first season for a couple of bucks, after which you could join Netflix briefly just to watch both seasons, then unsubscribe.

If some listeners know of it, or watch it and send feedback I'll look for it and try to share it.

Listener Feedback

Tom Crites

https://www.sciencedaily.com/releases/2025/12/251228020010.htm

A listener of ours, Tom Crites, sent SecurityNow! feedback containing nothing but a link. It was the subject of his note that caught my eye, which was: *"Vitamin D & Magnesium"*, and the link was to a just published piece on the Science Daily website. The piece was titled *"Why your vitamin D supplements might not be working."* Since I was unaware of a link between vitamin D and magnesium, since last week's holiday podcast was a re-play of our much earlier vitamin D podcast, and since magnesium is another substance that I have extensively researched and experimented with, I wanted to share the substance of this piece.

The summary at the top of the piece says:

> *A randomized trial from Vanderbilt-Ingram Cancer Center reveals that magnesium may be the missing key to keeping vitamin D levels in balance. The study found that magnesium raised vitamin D in people who were deficient while dialing it down in those with overly high levels—suggesting a powerful regulating effect. This could help explain why vitamin D supplements don't work the same way for everyone and why past studies linking vitamin D to cancer and heart disease have produced mixed results.*

The piece in Science News is a report on findings published in the The American Journal of Clinical Nutrition. It says:

> *The study, published in The American Journal of Clinical Nutrition, adds clarity to long-standing debates about vitamin D's links to colorectal cancer and other diseases. These questions have gained attention due to mixed results from major studies, including the VITAL trial. The new*

findings also reinforce earlier research from 2013 by the same team, which found that people with low magnesium intake often had low vitamin D levels as well.

Beyond confirming earlier observations, the trial uncovered an additional insight. Magnesium did not simply raise vitamin D across the board. Instead, it appeared to act as a regulator, lowering vitamin D levels in participants whose levels were already high. This is the first clinical evidence suggesting magnesium may help optimize vitamin D levels rather than just increase them, which could be important for reducing disease risk linked to vitamin D imbalance.

The Ingram Professor of Cancer Research and lead author of the study, explained that the healthiest vitamin D range appears to fall in the middle of a U-shaped curve. Previous observational studies have linked this middle range to the lowest risk of cardiovascular disease.

Despite earlier findings, vitamin D did not show a clear link to cardiovascular disease in the recent VITAL trial. Dai and co-author Martha Shrubsole, a research professor of Medicine in the Division of Epidemiology, are now examining whether magnesium could help explain these inconsistent results. Their work is part of the ongoing Personalized Prevention of Colorectal Cancer Trial.

Shrubsole said: "There's a lot of information being debated about the relationship between vitamin D and colorectal cancer risk that is based upon observational studies versus clinical trials. The information is mixed thus far."

The researchers turned their attention to magnesium after noticing that vitamin D supplements do not work equally well for everyone. Some people fail to raise their vitamin D levels even when taking high doses.

Dai said: "Magnesium deficiency shuts down the vitamin D synthesis and metabolism pathway."

The study included 250 adults considered at higher risk for colorectal cancer, either due to known risk factors or because they had previously had a precancerous polyp removed. Participants received either magnesium supplements or a placebo, with dosages tailored to their usual dietary intake.

Shrubsole noted that vitamin D insufficiency is widely recognized as a public health concern in the United States, and many patients are advised to take supplements based on blood test results. She said: "Vitamin D insufficiency is something that has been recognized as a potential health problem on a fairly large scale in the U.S. A lot of people have received recommendations from their health care providers to take vitamin D supplements to increase their levels based upon their blood tests. In addition to vitamin D, however, magnesium deficiency is an under-recognized issue. Up to 80 percent of people do not consume enough magnesium in a day to meet the recommended dietary allowance (RDA) based on those national estimates."

Shrubsole emphasized that magnesium intake in the study matched RDA guidelines and suggested that diet is the best way to increase magnesium levels. Foods rich in magnesium include dark leafy greens, beans, whole grains, dark chocolate, fatty fish such as salmon, nuts and avocados.

First , I want to acknowledge that I know this is not a health and nutrition podcast, and that as a health hobbyist and tinkerer with no formal medical training, I would never presume to be an authoritative source of medical information. So for those who have no interest in the topic of healthy longevity please rest assured that we will not be spending much time on the subject.

That said, the subject of the preservation and maintenance of health, vitality and energy as we age is an extreme personal passion of mine. It's something I've quietly devoted a large fraction of my life to researching and understanding. So in reply to this article which Tom brought to my attention I'm going to share a bit more of what I have learned and practiced.

Magnesium is a grossly underappreciated mineral. It is a required co-factor in more than 400 enzymatic reactions which transmute an organic molecule from one form to another. The book I read back in 2009 that started me down the path to understanding the role and importance of magnesium was called "The Magnesium Miracle", written by Carolyn Dean, MD and ND. I went over to Amazon to double-check the spelling of her name and Amazon flagged that book as having been purchased in 2009. It's $6 on Kindle and available in audio, Kindle and paperback.

The problem with magnesium, the reason for that report's observation that there's a general magnesium deficiency in the U.S., is that natural sources of magnesium have largely been removed from our lives. Before we obtain our water from municipal processing plants we used to drink water from wells or from river streams where the water would contain dissolved magnesium. And we'd be consuming plants that were rich sources of magnesium. But plants grown in magnesium-poor soil no longer provide the magnesium they once did. And the water we drink has been processed and filtered and chlorinated and bears little resemblance to the water that was consumed by pre-industrial man.

The upshot of living within a magnesium-poor environment is a magnesium-poor body that's unable to synthesize as many enzymes as it could if magnesium were available in greater supply.

The problem is how to get magnesium into us, because that turns out to be quite tricky. One of the things anyone who practices dietary supplementation comes to appreciate is that it can be difficult to get some substances into our bloodstream due to the fact that they must first survive our stomach acid's deliberately low acidic pH. And after surviving our stomach, the substance will be absorbed by our intestinal lining into our bloodstream, but its first destination will then be our livers where it may need to survive what's known as first-pass hepatic metabolism. Our livers may wish to take it apart and use its bits.

So what about magnesium? Our stomach's low pH acidic contents is the death of most forms of supplementary magnesium, at least as far as disassociation from its carrier atoms is concerned.

When my physician recommended that at my age I should start having periodic colonoscopy screenings, he handed me a large empty plastic jug. Well, it wasn't completely empty. There was a loose white power in the bottom of the jug. My instructions were to just add water and shake it up to dissolve the power. Then I was to pour a cup of this mixture every hour and drink it until the entire jug was empty. And not long after that, my entire intestinal tract would be similarly empty and I'd be ready to have my intestinal lining inspected for any abnormalities.

I'm sharing this seemingly off-topic story because that loose white power in the bottom of the initially empty jug was pure magnesium oxide. Magnesium oxide is the least expensive and least well absorbed of all magnesium formulations. It was what was traditionally used, along with ample water, to flush out one's intestines. My point is, this is not the magnesium you want to take if you're interested in replenishing and increasing your body's magnesium levels.

There are many forms of magnesium. There's magnesium oxide, magnesium citrate, magnesium malate, taurate, orotate and so on. All of these are simple salts of magnesium and they all have their proponents. Well, all except for probably magnesium oxide. It's inexpensive but it's essentially a laxative. And as you experiment you will find that magnesium, in general, has this effect. It's not harmful in itself, which is why it was once used by the medical establishment as the standard means of preparing a patient for scoping.

The key concept to understand is that the laxative effect induced by magnesium is the result of its nonabsorption into our bloodstream. Magnesium that remains in our intestines osmotically pulls water into the intestine which is not what we want for optimal health and digestion.

The problem is that to varying degrees all of those common simple salts of magnesium succumb to our stomach's acidic environment. Their molecules disassociate into their constituent atoms and they then suffer whatever fate awaits them.

The problem of effective dietary mineral supplement absorption was finally solved by a company called Albion Minerals. Their nutritional chemists came up with a means of sneaking magnesium and other minerals past our stomachs and getting them into our intestines without being broken apart by stomach acid. The key, instead of creating simple salts to carry the magnesium, was to bind magnesium into a dipeptide. It sounds more complicated than it is. A dipeptide is just two amino acids.

There are two forms of magnesium that are highly successful and are worth taking. One is known as Magnesium Glycinate Lysinate and other is Magnesium BisGlycinate. The first one, Magnesium Glycinate Lysinate consists of an atom of magnesium bound to the two amino acids glycine and lysine. Glycine is a good choice since it's the smallest of all amino acids and also because glycine is another substance that most people could use a lot more of. The second form of magnesium, Magnesium BisGlycinate is an atom of magnesium bound to a pair of glycine molecules. This is handy since glycine being the smallest of the aminos means that there's a higher percentage of elemental magnesium per milligram of the combined molecule.

The upshot of all this is that either of these dipeptide forms of magnesium will strongly resist disassociation in our low pH stomach environment. They will be able to transport the magnesium through our stomach and across our intestinal lining to carry it into our blood stream where it can be used by our body.

I should also note that unlike vitamin D and many other blood born substances whose levels **can** be checked with a blood test, there is no reliable blood test for magnesium because most magnesium is stored in our skeletal system where it is literally kept "out of circulation".

If you decide to get serious about magnesium, as I certainly have, the first thing I would recommend would be grabbing her book or otherwise learn much much more about why obtaining sufficient magnesium is important. I only just barely touched on the importance of this much under appreciated and inexpensive mineral for both immediate term and long term health.

Carolyn Dean and many others recommend that you experiment to find what's known as your gut tolerance level, that being the amount of magnesium you can consume in multiple divided daily doses of supplementation where you begin to notice a laxative effect, then back off from that. If you're taking one of the dipeptide forms that should initially be a lot of magnesium. But interestingly, after about nine months you may notice that you're no longer able to take that much. The reason is that you will have restored your body's depleted reservoirs and less is then needed to keep them topped off.

MongoBleed

The Mongo database's Wikipedia page explains:

MongoDB is a source-available, cross-platform, document-oriented database program. Classified as a NoSQL database product, MongoDB uses JSON-like documents with optional schemas. Released in February 2009 by 10gen (now MongoDB Inc.), it supports features like sharding, replication, and ACID transactions (from version 4.0). MongoDB Atlas, its managed cloud service, operates on AWS, Google Cloud Platform, and Microsoft Azure. Current versions are licensed under the Server Side Public License (SSPL). MongoDB is a member of the MACH Alliance.

As of May 2025, MongoDB was the 5th most popular database software. It focuses mostly on managing large databases of unstructured, "messy" data. It's typically used for mobile and web apps that commonly use unstructured databases. As of 2024, there were 50,000 MongoDB customers. MongoDB was originally best known as a NoSQL database product. The company released a database as-a-service product called Atlas in 2016 that became 70 percent of MongoDB's revenue by 2024. Over time, MongoDB added analytics, transactional databases, encryption, vector databases, ACID transactions, migration features, and other enterprise tools. Initially, the MongoDB software was free and open source under the AGPL license. MongoDB adopted an SSPL (server side public license) for future releases starting in 2018.

I've included a chart of the top five databases since I thought that our more DB-centric listeners might be curious about the industry's current database popularity lineup which has MongoDB in 5th place:

Rank			DBMS	Database Model	Score		
Jan 2026	Dec 2025	Jan 2025			Jan 2026	Dec 2025	Jan 2025
1.	1.	1.	Oracle	Relational, Multi-model	1237.34	+2.94	-21.42
2.	2.	2.	MySQL	Relational, Multi-model	867.52	-0.97	-130.63
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	706.26	-16.26	-92.30
4.	4.	4.	PostgreSQL	Relational, Multi-model	666.27	+6.84	+2.86
5.	5.	5.	MongoDB	Multi-model	376.74	+4.46	-25.77

To give us a quick snapshot of MongoDB's history, Wikipedia writes:

The American software company 10gen began developing MongoDB in 2007 as a component of a planned platform-as-a-service product. Two years later, in 2009, the company shifted to an open-source development model and began offering commercial support and other services. In 2013, 10gen changed its name to MongoDB Inc. On October 20th, 2017, MongoDB became a publicly traded company, listed on NASDAQ as MDB with an initial public offering (IPO) price of $24 per share. On November 8th, 2018, with the stable release 4.0.4, the software's license changed from AGPL 3.0 to SSPL. On October 30th, 2019, MongoDB teamed with Alibaba Cloud to offer Alibaba Cloud customers a MongoDB-as-a-service solution. Customers can use the managed offering from Alibaba's global data centers.

And the final item in Wikipedia's short summary of MongoDB's history reads:

> *In December 2025, **a major exploit was discovered** and titled "MongoBleed". This exploit led to the compromising of many corporate servers.*

And that final bit of news is, of course, the reason MongoDB is our main topic for this first podcast of 2026 ... because a major exploit it was and may still be since we know how slow software updating can be.

I've assembled the story of what happened from several sources, but I've chosen this not only because this is a new industry-wide mess, but also because the bug, which is now more than eight years old and thus is present in virtually all instances of MongoDB, is in many ways classic. No deep VooDoo was used. By the time we're finished here, I'm pretty certain that all of our listeners will clearly understand what happened, along with how and why.

What's being called "MongoBleed" is officially CVE-2025-14847, the CVE assigned to this recently discovered vulnerability affecting all versions of MongoDB since version 3.6 which was published on November 28th of 2017. This encompasses a huge span of major and minor releases. Essentially, a subtle bug was introduced into v3.6 a little over eight years ago, and it wasn't discovered until just over eight years later after everyone in the world had updated and upgraded to any of the past several years of releases.

Now, as for "everyone in the world", how "everyone" is that? The Internet scanning company Censys has identified on the order of ~87,000 publicly reachable MongoDB instances. And that's, of course, the crucial bit of information, since it's those publicly accessible instances that the bad guys have access to. And access they've had. This is one of those inopportunely timed events which became public just before Christmas and was not the Christmas present many IT workers were hoping to unwrap.

Exploitation of this long-present vulnerability allows an unauthenticated attacker – which is the fancy way the industry now refers to ***"anyone"*** – to read memory from the database server's heap, meaning anything that was allocated to memory from previous database operations. It's only the fact that this is not directly a remote code execution vulnerability that rendered this a CVSS of 8.7 rather than 9.8 or 10.0. And it's because this vulnerability leaks database server memory that it's been named "MongoBleed" which is meant to remind us of HeartBleed which was a flaw discovered in OpenSSL's v1.0.1 implementation.

So here comes a description of this exploit which just ruined many Christmases after bad guys figured out that they could spend their Christmas vacation reading out a bunch of MongoDB server data from around 87,000 publicly available server instances:

First of all, MongoDB uses its own TCP wire protocol instead of, for example, something like HTTP. This is not unusual for databases, especially when they're chasing high performance. So a general raw TCP connection is established to the server's TCP port 27017. As an aside, when I just asked ChatGPT which port MongoDB server uses, it helpfully added: *"Note: Exposing 27017 to the public internet is strongly discouraged; it should be firewalled or bound to private interfaces only."* Right. So even this unconscious LLM knows better than some 87,000 server deployments. Just to be clear, MongoDB itself probably never needs to be publicly exposed. It would normally be sitting ***behind*** a publicly-exposed web-app server, serving as that web-app server's backend database. MongoDB itself really doesn't have any public-exposure use cases. We've been talking a lot recently about the need to make these sorts of public exposure

mistakes far far more difficult to make. When I was swooning over Cisco's promises a month or two back it was because the noises they were making strongly suggested that this might have finally sunk in.

Anyway, so we connect with TCP to the server's port 27017. Mongo uses a binary variant of JSON called BSON. The request that's sent to the Mongo server contains one of these BSON messages and for speed of transmission that request can be compressed using ZLIB. Compression makes the message smaller, so one of the 32-bit values in the request header, which specifies that this request has been compressed, indicates the original decompressed size of the message. This allows the receiving MongoDB server to request the allocation of a block of memory from the underlying operating system into which MongoDB will decompress the message.

So an attacker creates and sends a server request which claims to contain far more data than it actually does. In response the server allocates the requested memory. An attacker might claim that the uncompressed request will require one megabyte when it only needs 1K. The critical flaw is that once MongoDB has finished decompressing it never checks the actual resulting size of the newly-decompressed payload. It trusts the data the user provided, using that as the actual size of the payload.

I need to stop here to hover over that phrase I used a bit longer. That phrase being: *"It trusts the data the user provided."* If we were produce a list of the root causes behind many of the worst flaws that have been found in software, *"trusting user-provided input"* would definitely be right up there near the top – if not in first place – since even buffer overflows typically result from the similar mistake of trusting and using something that a malicious user deliberately provided. In this case we have a deliberate buffer ***underflow*** that results entirely from trusting input from the user. Okay. So what's the big deal about allocating an oversize buffer that's not used?

In many contemporary languages, memory allocated for a program is cleared to zeroes before it's returned for use by the caller. But malloc(), the memory allocation function used by C and C++ doesn't bother doing so. This is part of the trusting, performance-oriented and dangerous legacy of C. Since zeroing RAM takes time and blows the processor cache, C deliberately returns uninitialized memory. And wouldn't you know, MongoDB is written in C++. The result of the bug is that multiple megabytes of the server's raw internal data can be exfiltrated to the attacker. This data might and often does contain cleartext passwords and credentials, session tokens, API keys, customer data, database configurations and system info, docker paths and client IP addresses, and so on. In short, all of the internal operations of the server that would otherwise never be made available to anyone.

So let's sum this up: An attacker sends an otherwise valid MongoDB message which indicates that it employs compression. But that compressed message is deliberately manipulated to specify a hugely exaggerated claim about the message's uncompressed size. Since the server has no way to know any better in advance, MongoDB obtains a large and uninitialized buffer from the C runtime based upon the attacker's message's claimed need. MongoDB's built-in Zlib decompresses the much smaller compressed data into just the front of the huge decompression buffer, thus avoiding overwriting the motherload of data that's already be there. Subsequent commands then instruct the database server to return to this attacker what it believes is the user's provided data even though it's actually megabytes of whatever data had been previously used and left behind by other database operations of any kind.

It's obvious now why this critical flaw was nicknamed "MongoBleed", and also why it was given a CVSS of 8.7. Although it doesn't allow a remote attacker to execute their own code on the

server, it's a data exfiltration flaw that's just about as bad as it gets. Proof of concept code has been published on Github and the flow is trivial to exploit. There's nothing like *"it only works less than one time in one thousand and only when your code wins some slippery internal race condition."* Nope, this one is extremely straightforward, and the attacker receives much more than a tiny trickle of data. Without raising any alarms, crashing the server or otherwise calling attention to itself, the abuse of this longstanding vulnerability, that's been present in every version of MongoDB published in the past eight years, allows remote bad guys to freely rummage around inside the more than 87,000 currently online and publicly exposed instances of MongoDB. They're able to keep sucking out and examining megabytes of a server's data that is assumed to be utterly private, and which might therefore contain very juicy information.

It's always fun to see Kevin Beaumont's take on these things. On December 26th, Kevin posted:

> *Somebody from Elastic Security decided to post an exploit for CVE-2025–14847 on Christmas Day. The vuln, which dropped just before Christmas, in theory allowed memory read without authentication. Patches are available. It impacts every version of MongoDB going back about a decade. Another vendor decided it would be a great idea to post technical details on Christmas Eve:*
>
> https://www.ox.security/blog/attackers-could-exploit-zlib-to-exfiltrate-data-cve-2025-14847/
>
> *The exploit dropped yesterday and is the first public exploit. It's dubbed MongoBleed. I've validated that said exploit is real, you can just supply an IP address of a MongoDB instance and it'll start ferreting out in-memory things such as database passwords (which are plain text), AWS secret keys etc. The exploit specifically looks for those class of credentials and secrets, too. The internet footprint of MongoDB is very large, over 200k instances. Because of how simple this is now to exploit — the bar is removed — expect high likelihood of mass exploitation and related security incidents. The exploit author has provided no details on how to detect exploitation in logs via products like.. Elastic. Advice would be to keep calm and patch internet facing assets.*

So now we know all about this mess. And Kevin's ending advice to keep calm and patch Internet facing assets reminded me of something Leo and I talked about long ago.

We made the observation, many times in fact, that once a user's system had been infected by something, anything, it was really never again possible to trust it. How could anyone ever know with 100% assurance that every last bit of an infection had been removed? And what about whether an infection might have spread over the local network to also infect other assets? In short, it's a mess.

We've also seen instances where huge problems resulted when companies didn't take prior intrusions seriously enough. The advice is always to *"rotate all credentials may have had any chance of being exposed"* – meaning invalidate any long-term authentication tokens, change all passwords, and so on. But as I said, we've seen instances where companies, for one reason or another – oversight? Laziness? Lack of belief it was really necessary? Who knows? But for whatever reason they failed to adequately and fully remediate the consequences of a breach only to suffer again and often even worse later.

So now consider the plight of corporate users of publicly-exposed MongoDB servers. You're told that for the past eight years, the database server you've been relying upon has contained a flaw that allows for effectively unfettered mass exfiltration of that server's internal working memory which contains myriad private credentials, past database search results, and essentially any and

all proprietary information to which that server may have had access or may have been stored and retrieved by it over time. To call this a mess is truly an understatement. And this mess is now squarely in the laps of every enterprise that was using an publicly exposed MongoDB server.

My question is, why was even a single instance of MongoDB publicly exposed? I'm sitting here, as I talk to Leo and our audience, in Southern California. From my location here, I have access to any and all of those 87,000-some instances of MongoDB. Why? Why do I have access? Why can I send out a TCP SYN packet to port 27017 of any of those 87,000 IPs and promptly receive a TCP SYN/ACK packet inviting me to complete the TCP connection? Why? I have no need to ever do so. Whoever runs that MongoDB instance certainly doesn't want or expect me, sitting here in Southern California to be able to connect to their database server. But I can. Why?

By now I hope that everyone in this podcast's audience understands not only that this is wrong, but just how wrong it is.

If I were to confront whomever it was who set up any given instance among those 87,000, that IT person would probably respond: *"Well, we've password protected access to our database and you can't do anything without that."* Oh yeah? MongoBleed baby! No authentication needed.

The decompression of the message is pre-authentication and never requires any form of authentication for its exploitation. One of the refrains everyone listening to this podcast has been hearing from me, beginning last year when it finally so clearly crystallized after we all witnessed mistake after mistake after mistake which all carried the same pattern – this pattern – which is that authentication does not work.

Now the world depends upon and turns on the strength of authentication. So I obviously don't mean that it can't work. What I mean is that it cannot be absolutely depended upon to work. In my hypothetical conversation with that MongoDB IT person, their defense of their database's utterly unnecessary public exposure was that I didn't know the secret handshake, so they didn't feel the need to take every possible precaution. The massive sweep of today's MongoBleed vulnerability is the direct consequence of that way of thinking. That way of thinking is obviously defective and wrong.

Sitting here in Southern California, I have no need to be able to connect to any of those 87,000 MongoDB servers even if only to test the strength of their authentication. I should not be allowed to do that. But I can. And that's on them. That's on each and every one of them individually.

This erroneous reliance upon remote authentication – which we keep seeing over and over does not work – is perhaps the single most important thing that has to change in today's Internet-worked world. And what's most galling is that it's not about flaws or mistakes. It's entirely about policy and caring. If we cared to, we could fix it.